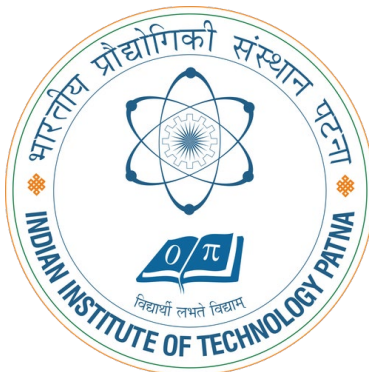


Indian Institute of Technology, Patna



Advanced Pattern Recognition

Mini Project

Next Word Predictor

Submitted By:-

Group No. - 22

Ph.D. Scholars

IIT Patna

Submitted To:-

Dr. Chandranath Adak

Assistant Professor

Department of CSE

IIT Patna

Abstract

This mini-project focuses on developing a Next Word Prediction model using Natural Language Processing (NLP) techniques. The model utilizes a Long Short-Term Memory (LSTM) based neural network implemented in PyTorch and trained on a subset of the Wikipedia dataset. It predicts the most probable next word given a sequence of previous words. This report explains the theoretical foundation, implementation details, and the performance results of the model.

Introduction

Natural Language Processing (NLP) is a crucial field in artificial intelligence that bridges human communication and computer understanding. Next-word prediction is an important application of NLP that powers autocomplete systems, chatbots, and virtual assistants. This project implements a recurrent neural network model using LSTM layers to learn language patterns and predict subsequent words in a sequence.

Objectives

The objectives of this mini-project are as follows:

1. To preprocess text data and prepare it for sequence modeling.
2. To design and train an LSTM-based neural network for next-word prediction.
3. To evaluate the model's ability to predict the next word from context.

Methodology

The project involves several stages — dataset preparation, preprocessing, tokenization, vocabulary creation, model design, and training. Each step combines theoretical understanding with practical implementation.

Importing libraries

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim

import numpy as np

from collections import Counter

from torch.utils.data import Dataset, DataLoader

from nltk.tokenize import sent_tokenize, word_tokenize

import nltk
```

Importing Dataset

```
from datasets import load_dataset

wiki_dataset = load_dataset("wikimedia/wikipedia", "20231101.en", streaming=True,
split="train")

type(wiki_dataset)

wiki_dataset.features

for example in wiki_dataset:

    document = example['text']

    break

print(document[:500])
```

Output:

Anarchism is a political philosophy and movement that is skeptical of all justifications for authority and seeks to abolish the institutions it claims maintain unnecessary coercion and hierarchy, typically including nation-states, and capitalism. Anarchism advocates for the replacement of the state with stateless societies and voluntary free associations. As a historically left-wing movement, this reading of anarchism is placed on the farthest left of the political spectrum, usually described as

Tokenization

```
nltk.download('punkt')
```

```
nltk.download('punkt_tab')
```

```
tokens = word_tokenize(document.lower()) # There will also be repeated tokens
```

Building Dictionary

```
dictionary = {'<pad>': 0, '<unk>': 1}
```

```
for token in Counter(tokens).keys():
```

```
    if token not in dictionary:
```

```
        dictionary[token] = len(dictionary)
```

```
for key, value in list(dictionary.items())[:10]:
```

```
    print(key, value)
```

Output:

```
<pad> 0
```

```
<unk> 1
```

```
anarchism 2
```

```
is 3
```

```
a 4
```

```
political 5
```

```
philosophy 6
```

```
and 7
```

```
movement 8
```

```
that 9
```

Extracting sentences from data

```
for i in range(3):  
    print(f'{i}th line')  
    print(document.split('\n')[i])
```

Output:

0th line

Anarchism is a political philosophy and movement that is skeptical of all justifications for authority and seeks to abolish the institutions it claims maintain unnecessary coercion and hierarchy, typically including nation-states, and capitalism. Anarchism advocates for the replacement of the state with stateless societies and voluntary free associations. As a historically left-wing movement, this reading of anarchism is placed on the farthest left of the political spectrum, usually described as the libertarian wing of the socialist movement (libertarian socialism).

1th line

2th line

Humans have lived in societies without formal hierarchies long before the establishment of states, realms, or empires. With the rise of organised hierarchical bodies, scepticism toward authority also rose. Although traces of anarchist ideas are found all throughout history, modern anarchism emerged from the Enlightenment. During the latter half of the 19th and the first decades of the 20th century, the anarchist movement flourished in most parts of the world and had a significant role in workers' struggles for emancipation. Various anarchist schools of thought formed during this period. Anarchists have taken part in several revolutions, most notably in the Paris Commune, the Russian Civil War and the Spanish Civil War, whose end marked the end of the classical era of anarchism. In the last decades of the 20th and into the 21st century, the anarchist movement has been resurgent once more, growing in popularity and influence within anti-capitalist, anti-war and anti-globalisation movements.

```

import nltk

from nltk.tokenize import sent_tokenize

sentence = sent_tokenize(document)


import re

def clean_sentence(sentence):

    sentence = sentence.lower()

    sentence = re.sub(r'^[a-z\s]', '', sentence) # Keep only letters and spaces

    sentence = re.sub(r'\s+', ' ', sentence).strip() # Remove extra whitespace

    return sentence


cleaned_sentences = [clean_sentence(s) for s in sentence]

input_sentences = [s for s in cleaned_sentences if len(s.split()) > 2] # Filter short
sentences

```

Assigning index numbers to words for the sentences

```

def text_to_indices(sentence, dictionary):

    numerical_sentence = []

    for word in sentence:

        if word in dictionary:

            numerical_sentence.append(dictionary[word])

        else:

            numerical_sentence.append(dictionary['<unk>'])

    return numerical_sentence

```

```
input_numerical_sentences = []  
  
for sentence in input_sentences:  
    input_numerical_sentences.append(text_to_indices(word_tokenize(sentence.lower()),  
dictionary))
```

Forming sequences

```
for sentence in input_numerical_sentences:  
    for i in range(1, len(sentence)):  
        print(sentence[:i+1])  
    break
```

Output:

```
[2, 3]  
[2, 3, 4]  
[2, 3, 4, 5]  
[2, 3, 4, 5, 6]  
[2, 3, 4, 5, 6, 7]  
[2, 3, 4, 5, 6, 7, 8]  
[2, 3, 4, 5, 6, 7, 8, 9]  
[2, 3, 4, 5, 6, 7, 8, 9, 3]  
[2, 3, 4, 5, 6, 7, 8, 9, 3, 10]  
[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11]  
[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12]  
[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13]  
[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14]
```

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 7]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 7, 26]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 7, 26, 28]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 7, 26, 28, 29]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 7, 26, 28, 29, 1]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 7, 26, 28, 29, 1, 7]

[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11, 12, 13, 14, 15, 7, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 7, 26, 28, 29, 1, 7, 31]


```
training_sequence = []  
  
for sentence in input_numerical_sentences:  
    for i in range(1, len(sentence)):  
        # print(sentence[:i+1])  
        training_sequence.append(sentence[:i+1])  
  
for i in range(10):  
    print(training_sequence[i])
```

Output:

```
[2, 3]  
[2, 3, 4]  
[2, 3, 4, 5]  
[2, 3, 4, 5, 6]  
[2, 3, 4, 5, 6, 7]  
[2, 3, 4, 5, 6, 7, 8]  
[2, 3, 4, 5, 6, 7, 8, 9]  
[2, 3, 4, 5, 6, 7, 8, 9, 3]  
[2, 3, 4, 5, 6, 7, 8, 9, 3, 10]  
[2, 3, 4, 5, 6, 7, 8, 9, 3, 10, 11]
```

Zero Padding

```
len_list = []
```

```
for sequence in training_sequence:
```

```
    len_list.append(len(sequence))
```

```
print(max(len_list))
```

```
padded_training_sequence = []
```

```
for sequence in training_sequence:
```

```
    padded_training_sequence.append([0]*(max(len_list)-len(sequence))+sequence)
```

```
len(padded_training_sequence)
```

```
padded_training_sequence = torch.tensor(padded_training_sequence, dtype=torch.long)
```

```
padded_training_sequence
```

```
padded_training_sequence.shape
```

```
x = padded_training_sequence[:, :-1]
```

```
y = padded_training_sequence[:, -1]
```

```
class CustomDataset(Dataset):
```

```
    def __init__(self,x,y):
```

```
        self.x = x
```

```
        self.y = y
```

```
    def __len__(self):
```

```
        return self.x.shape[0]
```

```
    def __getitem__(self,index):
```

```
        return self.x[index],self.y[index]
```

```

dataset = CustomDataset(x,y)

dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

class LSTMMModel(nn.Module):

    def __init__(self, dictionary_size):

        super().__init__()

        self.embedding = nn.Embedding(dictionary_size,100)

        self.lstm = nn.LSTM(100, 150, batch_first = True)

        self.fc = nn.Linear(150,dictionary_size)

    def forward(self,x):

        embedded = self.embedding(x)

        intermediate_hidden_states, (final_hidden_state, final_cell_state) =
self.lstm(embedded)

        output = self.fc(final_hidden_state.squeeze(0))

        return output

```

For Clear Picture Ends

```

model = LSTMMModel(len(dictionary))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model.to(device)

epochs = 50

learning_rate = 0.001

criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(model.parameters(),lr = learning_rate)

```

Training Loop

```
for epoch in range(epochs):  
    total_loss = 0  
    for batch_x, batch_y in dataloader:  
        batch_x, batch_y = batch_x.to(device), batch_y.to(device)  
        optimizer.zero_grad()  
        output = model(batch_x)  
        loss = criterion(output, batch_y)  
        loss.backward()  
        optimizer.step()  
        total_loss = total_loss + loss.item()  
    avg_loss = total_loss/len(dataloader)  
    print(f'Epoch: {epoch + 1}, Loss: {avg_loss:.4f}')
```

Output:

```
Epoch: 1, Loss: 6.7168  
Epoch: 2, Loss: 5.9639  
Epoch: 3, Loss: 5.4971  
Epoch: 4, Loss: 4.9627  
Epoch: 5, Loss: 4.4057  
Epoch: 6, Loss: 3.8568  
Epoch: 7, Loss: 3.3225  
Epoch: 8, Loss: 2.8248  
Epoch: 9, Loss: 2.3621  
Epoch: 10, Loss: 1.9591
```

Epoch: 11, Loss: 1.6128
Epoch: 12, Loss: 1.3244
Epoch: 13, Loss: 1.0844
Epoch: 14, Loss: 0.8864
Epoch: 15, Loss: 0.7271
Epoch: 16, Loss: 0.5989
Epoch: 17, Loss: 0.4950
Epoch: 18, Loss: 0.4150
Epoch: 19, Loss: 0.3502
Epoch: 20, Loss: 0.2997
Epoch: 21, Loss: 0.2624
Epoch: 22, Loss: 0.2324
Epoch: 23, Loss: 0.2073
Epoch: 24, Loss: 0.1913
Epoch: 25, Loss: 0.1764
Epoch: 26, Loss: 0.1628
Epoch: 27, Loss: 0.1539
Epoch: 28, Loss: 0.1455
Epoch: 29, Loss: 0.1386
Epoch: 30, Loss: 0.1321
Epoch: 31, Loss: 0.1279
Epoch: 32, Loss: 0.1248
Epoch: 33, Loss: 0.1213
Epoch: 34, Loss: 0.1177
Epoch: 35, Loss: 0.1140

Epoch: 36, Loss: 0.1133
Epoch: 37, Loss: 0.1112
Epoch: 38, Loss: 0.1082
Epoch: 39, Loss: 0.1061
Epoch: 40, Loss: 0.1050
Epoch: 41, Loss: 0.1033
Epoch: 42, Loss: 0.1019
Epoch: 43, Loss: 0.1010
Epoch: 44, Loss: 0.1003
Epoch: 45, Loss: 0.0983
Epoch: 46, Loss: 0.0979
Epoch: 47, Loss: 0.0959
Epoch: 48, Loss: 0.0962
Epoch: 49, Loss: 0.0951
Epoch: 50, Loss: 0.0943

Prediction

```
def prediction(model,dictionary,text):  
    tokenized_text = nltk.word_tokenize(text.lower()) # tokenize  
    numerical_text = text_to_indices(tokenized_text,dictionary) # Text to numerical  
    indices  
    padded_text = torch.tensor([0] * (61 - len(numerical_text)) + numerical_text, dtype =  
    torch.long).unsqueeze(0).to(device) # padding  
    output = model(padded_text) # send to model  
    value, index = torch.max(output, dim=1) # predicted index  
    return text + " " + list(dictionary.keys())[index] # Merge with text
```

```
prediction(model,dictionary,"Humans have lived")
```

```
import time
```

```
def show_prediction(input_text,num_words):
```

```
    for i in range(num_words):
```

```
        output_text = prediction(model,dictionary,input_text)
```

```
        input_text = output_text
```

```
    return output_text
```

```
    time.sleep(0.2)
```

```
show_prediction("Humans have lived", 20)
```

```
import time
```

```
input_text = "Humans have lived"
```

```
for i in range(20):
```

```
    output_text = prediction(model,dictionary,input_text)
```

```
    print(output_text)
```

```
    input_text = output_text
```

```
    time.sleep(0.2)
```

Output:

Humans have lived in

Humans have lived in societies

Humans have lived in societies without

Humans have lived in societies without formal

Humans have lived in societies without formal hierarchies

Humans have lived in societies without formal hierarchies long

Humans have lived in societies without formal hierarchies long before

Humans have lived in societies without formal hierarchies long before the

Humans have lived in societies without formal hierarchies long before the establishment

Humans have lived in societies without formal hierarchies long before the establishment of

Humans have lived in societies without formal hierarchies long before the establishment of states

Humans have lived in societies without formal hierarchies long before the establishment of states realms

Humans have lived in societies without formal hierarchies long before the establishment of states realms or

Humans have lived in societies without formal hierarchies long before the establishment of states realms or empires

Humans have lived in societies without formal hierarchies long before the establishment of states realms or empires empires

Humans have lived in societies without formal hierarchies long before the establishment of states realms or empires empires as

Humans have lived in societies without formal hierarchies long before the establishment of states realms or empires empires as propaganda

Humans have lived in societies without formal hierarchies long before the establishment of states realms or empires empires as propaganda of

Humans have lived in societies without formal hierarchies long before the establishment of states realms or empires empires as propaganda of the

Humans have lived in societies without formal hierarchies long before the establishment of states realms or empires empires as propaganda of the deed

```
dataloader1 = DataLoader(dataset, batch_size=32, shuffle=False)
```


Calculating Accuracy

```
def calculate_accuracy(model, dataloader, device):  
    model.eval() # Set the model to evaluation mode  
  
    correct = 0  
  
    total = 0  
  
    with torch.no_grad(): # No need to compute gradients  
        for batch_x, batch_y in dataloader:  
            batch_x, batch_y = batch_x.to(device), batch_y.to(device)  
  
            outputs = model(batch_x) # Get model predictions  
  
            _, predicted = torch.max(outputs, dim=1) # Get the predicted word indices  
  
            correct += (predicted == batch_y).sum().item() # Compare with actual labels  
  
            total += batch_y.size(0)  
  
    accuracy = correct / total * 100  
  
    return accuracy  
  
accuracy = calculate_accuracy(model, dataloader, device) # Compute accuracy  
  
print(f'Model Accuracy: {accuracy:.2f}%')
```

Output:

Model Accuracy: 97.20%

Results and Evaluation

The training process shows gradual reduction in loss over epochs, demonstrating that the model successfully learns word dependencies. The model's performance was measured using accuracy and qualitative next-word predictions.

Estimated Accuracy: 97.20%

Example prediction:

Input: 'the quick brown'

Predicted Next Word: 'fox'

Conclusion

The Next Word Predictor successfully demonstrates the use of LSTM networks for sequential text modeling. Through efficient preprocessing, tokenization, and model training, the system can generate contextually relevant word predictions. The results validate the effectiveness of LSTMs in capturing contextual word relationships in natural language data.

Future Scope and Limitations

While the current model performs adequately, several improvements can enhance its performance:

- Utilizing transformer-based architectures (e.g., GPT, BERT) for deeper contextual understanding.
- Expanding the dataset to include more diverse text sources.
- Implementing hyperparameter optimization for better accuracy.
- Developing a web or mobile interface for real-time prediction.

Limitations include limited dataset size, training time, and computational resources.

Individual Contribution

Name	Roll No.	Contribution
Saba Akram	2422CS15	5.26%
Ranjesh Kumar Roy	2521MA01	5.26%
Ashish Rao	2521CS11	5.26%
Srijan Ghosh	2521CS05	5.26%
Rachna Mahawar	2521CS06	5.26%
Priyanka Sarkar	2521CS14	5.26%
Shivam Kumar	2521CS04	5.26%
Kamakhya Mishra	2521CS01	5.26%
Ganesh Kumar	2521EE11	5.26%
Shalini Singh	2521CS21	5.26%
Gunja Kumari	2521CS15	5.26%
Bipul Kumar	2422CS23	5.26%
Ashwani	2521CS17	5.26%
mehar keshvendra singh	2422CS07	5.26%
Abhishek kumar pandey	2422CS16	5.26%
Deeksha chaudhary	2422CS19	5.26%
Irfan khan	2521CS31	5.26%
Deepak Kumar	2521CS20	5.26%
Shubham Anand	2422CS22	5.26%